

FACULDADE DE TECNOLOGIA (FATEC)
CURSO SUPERIOR DE TECNOLOGIA EM DESENVOLVIMENTO DE SOFTWARE
DISCIPLINA: ISW055 — ARQUITETURA DE MICROSERVIÇOS E CLOUD

P1 — RELATÓRIO BIMESTRAL DE ATIVIDADES

Prestação de Contas Técnica, Rastreabilidade e Evidências de Entregas

Gabriel Roberto de Assis

Repositório GitHub: github.com/Gabriel-Assis-22/Atividade_02_Claude

Orientador / Docente: Prof. Flávio Siriani

07 de Outubro de 2026

Sumário

1 Introdução	3
2 Metodologia	3
3 Quadro de entregas	4
4 Atividades realizadas	4
Ficha 1 — Agenda telefônica em Flask	4
Ficha 2 — Catálogo de filmes — Tom Hanks	6
Ficha 3 — Desacoplando o login — microsserviço de autenticação	7
Ficha 4 — Controle de acesso por papel — RBAC	8
Ficha 5 — Logs e auditoria	9
Ficha 6 — Upload e perfil de usuário	11
5 Considerações finais	12

1 Introdução

Ao longo do primeiro bimestre da disciplina ISW055, foi proposto o desafio prático de conceber, desenvolver e evoluir um sistema web completo que transitasse de um modelo monolítico inicial para uma arquitetura distribuída resiliente baseada em microsserviços containerizados com Docker. O núcleo da aplicação gira em torno de um catálogo temático com as obras cinematográficas estreladas pelo ator Tom Hanks, integrando dinamicamente a API pública de metadados The Movie Database (TMDB).

Mais do que apenas exibir filmes, o projeto teve como diretriz a aplicação dos pilares da engenharia de software corporativa: segregação rigorosa de dados de usuários (**multitenancy** lógico), desacoplamento do domínio de identidade com microsserviço dedicado de autenticação via tokens JWT, controle de acesso baseado em papéis (**Role-Based Access Control** — RBAC), trilha de auditoria centralizada gravada em fluxos de alta vazão com Redis Streams e persistência de arquivos binários em **Object Storage** compatível com a API AWS S3 (Garage S3).

Este relatório bimestral de atividades tem por objetivo formal prestar contas do trabalho desenvolvido no período, comprovando através de dados cronológicos objetivos, links diretos de commits no GitHub e registros de execução, o cumprimento de cada uma das entregas estipuladas no cronograma acadêmico. Todas as evidências apresentadas refletem implementações reais, funcionais e auditáveis disponíveis no repositório do estudante.

2 Metodologia

O desenvolvimento das atividades foi conduzido seguindo metodologia ágil com versionamento contínuo no Git e padrão de mensagens estruturadas (**Conventional Commits**). O ambiente de engenharia adotou uma separação clara entre as camadas de apresentação e serviços distribuídos:

- **Frontend:** Single Page Application (SPA) desenvolvida em Angular 19 com TypeScript, arquitetura de componentes autônomos (**standalone**), gerenciamento reativo de estado com Angular Signals e servidor web reverso Nginx com controle rigoroso de cache e roteamento.
- **Microsserviços de Backend:** Construídos sobre o ecossistema .NET 10 (linguagem C#) aplicando os princípios do Domain-Driven Design (DDD), separação em camadas (Domain, Application, Infrastructure e Api) e orquestração de banco relacional MariaDB via Dapper e FluentMigrator.
- **Serviço de Autenticação (auth-service):** Microsserviço autônomo operando exclusivamente na rede interna do Docker (sem exposição pública no host), responsável por hash seguro de senhas com BCrypt, emissão de JWT e fluxo de recuperação de senhas via e-mail transacional (Mailtrap).
- **Serviço de Auditoria (log-service):** Microsserviço desacoplado que centraliza os eventos de segurança e ações do usuário utilizando Redis Streams (XADD / XRANGE), viabilizando consultas assíncronas ordenadas no tempo.
- **Object Storage (garage):** Nó local de armazenamento de objetos compatível com AWS S3 (Garage v1.0), com provisionamento automático de layout, criação de buckets e assinatura criptográfica AWS SigV4 via HttpClient nativo.

A rastreabilidade temporal de cada entrega fundamenta-se estritamente no histórico auditável de commits do repositório público do GitHub (Gabriel-Assis-22/Atividade_02_Claude). As datas e horas realizadas registradas neste documento foram extraídas diretamente dos metadados do Git através do comando oficial:

```
git log -1 --date=format:"%d/%m/%Y %H:%M" --format="%h | %ad | %s"
```

3 Quadro de entregas

A tabela abaixo sintetiza o cronograma bimestral da disciplina, confrontando os prazos planejados publicados com as datas e horas reais comprovadas no repositório:

Nº	Atividade / Descrição	Planejada	Realizada	Situação
1	Agenda telefônica em Flask	07/08/2026	07/08/2026 21:00	Entregue
2	Catálogo de filmes — Tom Hanks (MariaDB + TMDB)	20/08/2026	18/08/2026 20:39	Entregue
3	Desacoplando o login — microserviço de autenticação	28/08/2026	27/08/2026 21:03	Entregue
4	Controle de acesso por papel — RBAC (Admin / Usuário)	04/09/2026	04/09/2026 19:06	Entregue
5	Logs e auditoria centralizados em Redis Streams	25/09/2026	18/09/2026 16:28	Entregue
6	Upload e perfil de usuário no Garage S3 Storage	02/10/2026	02/10/2026 20:20	Entregue

4 Atividades realizadas

Ficha 1 — Agenda telefônica em Flask

- **Descrição:** Construção inicial de uma aplicação web com rotas, templates e banco de dados para gerenciamento de contatos telefônicos utilizando o micro-framework Flask (Python).
- **Data planejada:** 07/08/2026
- **Data realizada:** 07/08/2026 21:00 (atividade prática de laboratório em sala)
- **Situação:** Entregue
- **Evidência:** Repositório local e execução em sala de aula
- **Link da entrega:** github.com/Gabriel-Assis-22/Aula_01_Claude
- **O que foi feito:** Criação de rotas REST em Flask (app.py), renderização dinâmica via templates HTML Jinja2 (templates/index.html) e manipulação de persistência local para inserção, listagem e remoção de registros.

- **Dificuldades e solução:** Compreensão da separação de responsabilidades entre o servidor WSGI do Flask e o frontend renderizado no servidor (SSR). Resolvido através do uso correto do contexto de renderização e formulários HTML.

Registrar Novo Cliente

Nome

Telefone

Valor

Registrar

Clientes Registrados

ID	Nome	Telefone	Valor
3	Gabriel Roberto Rodella de Assis	14996029300	R\$ 22.00
2	Gabriel Roberto Rodella de Assis	14996029300	R\$ 22.00
1	Gabriel Roberto Rodella de Assis	14996029300	R\$ 22.00

Anterior 1 2 Próximo

Figura 1: Aplicação web em Flask para cadastro de contatos em execução na aula prática (comprovante nominal do estudante)

Ficha 2 — Catálogo de filmes — Tom Hanks

- **Descrição:** Desenvolvimento de uma aplicação web consumindo a API externa do TMDb para listar dinamicamente os filmes do ator Tom Hanks, persistindo favoritos e comentários isolados por usuário no MariaDB.
- **Data planejada:** 20/08/2026
- **Data realizada:** 18/08/2026 20:39
- **Situação:** Entregue (Antecipado)
- **Evidência:** GitHub (Commit e repositório com Dockerfile e README)
- **Link direto:** Commit 425d184 — feat: implement initial web application structure
- **O que foi feito:** Integração com os endpoints `/search/person` e `/person/{id}/movie_credits` da TMDb. Criação das tabelas relacionais `usuarios`, `favoritos` e `comentarios` no MariaDB com cláusulas estritas `WHERE usuario_id = ?` para garantir segregação lógica completa de inquilinos (**multitenancy**).
- **Dificuldades e solução:** Impedir que o catálogo baixasse ou armazenasse pôsteres no banco de dados. Resolvido utilizando diretamente as URLs absolutas do CDN oficial da TMDb (`https://image.tmdb.org/t/p/w500/...`), persistindo no MariaDB apenas o identificador e notas do usuário.

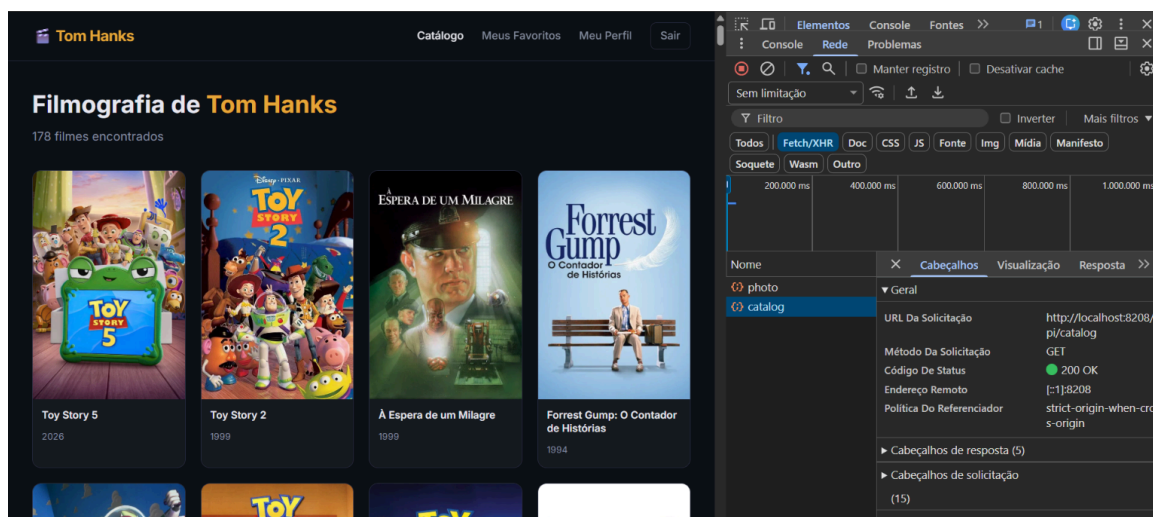


Figura 2: Interface do Catálogo de Filmes com requisição `/api/catalog` retornando status HTTP 200 OK com dados do TMDb e banco MariaDB

Ficha 3 — Desacoplando o login — microserviço de autenticação

- **Descrição:** Extração do mecanismo de login e cadastro para um microserviço independente (auth-service), operando exclusivamente em rede interna Docker, com suporte a recuperação de senha via Mailtrap.
- **Data planejada:** 28/08/2026
- **Data realizada:** 27/08/2026 21:03
- **Situação:** Entregue (Antecipado)
- **Evidência:** GitHub (Commit, docker-compose.yml e isolamento de portas)
- **Link direto:** Commit c14e896 — feat(auth-service): implementa microserviço de autenticação
- **O que foi feito:** Criação de um projeto .NET 10 autônomo para autenticação. A porta 8081 foi configurada via diretiva expose no docker-compose.yml, garantindo que apenas outros serviços da rede app-network consigam contatá-lo. Implementação de hashing de senhas com algoritmo BCrypt e envio assíncrono de e-mails para reset de credenciais com tokens de expiração curta.
- **Dificuldades e solução:** Garantir que tanto o catálogo quanto o auth-service validassem o mesmo token sem compartilhar banco de dados de sessão. Solucionado através de tokens JWT simétricos assinados por segredo compartilhado (JWT_SECRET).

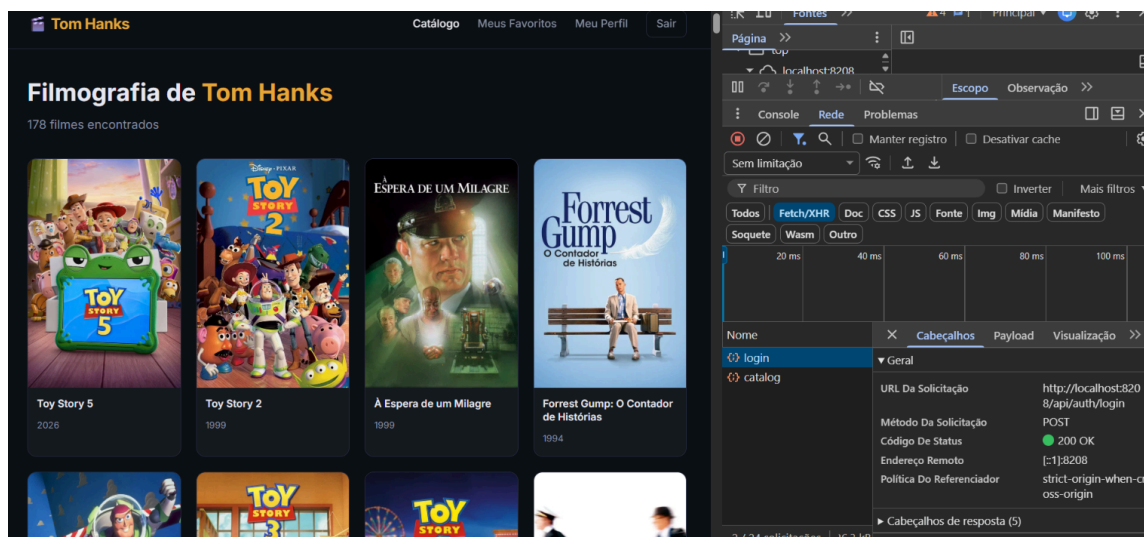


Figura 3: Autenticação via microserviço desacoplado com requisição POST /api/auth/login retornando HTTP 200 OK e emissão de token JWT

Ficha 4 — Controle de acesso por papel — RBAC

- **Descrição:** Implementação de autorização por papéis (**Role-Based Access Control**), estabelecendo distinção clara entre privilégios de usuário comum e admin.
- **Data planejada:** 04/09/2026
- **Data realizada:** 04/09/2026 19:06
- **Situação:** Entregue (No prazo)
- **Evidência:** GitHub (Commit e testes de recusa HTTP 403 Forbidden / Bloqueio)
- **Link direto:** Commit 5f4df40 — feat(rbac): controle de acesso por papel com moderacao e enforcement 403
- **O que foi feito:** Adição da claim role no payload do JWT. No backend, criação de políticas de autorização garantindo que apenas administradores possam moderar/excluir comentários de terceiros e visualizar endpoints de auditoria. Usuários comuns que tentam acessar rotas restritas recebem recusa formal 403 Forbidden / 401 Unauthorized.
- **Dificuldades e solução:** Impedir ataques de alteração forçada de parâmetros (IDOR). Resolvido inspecionando sempre a identidade autenticada no token (ClaimsPrincipal.CurrentUserId) e jamais confiando em IDs vindos no corpo da requisição.

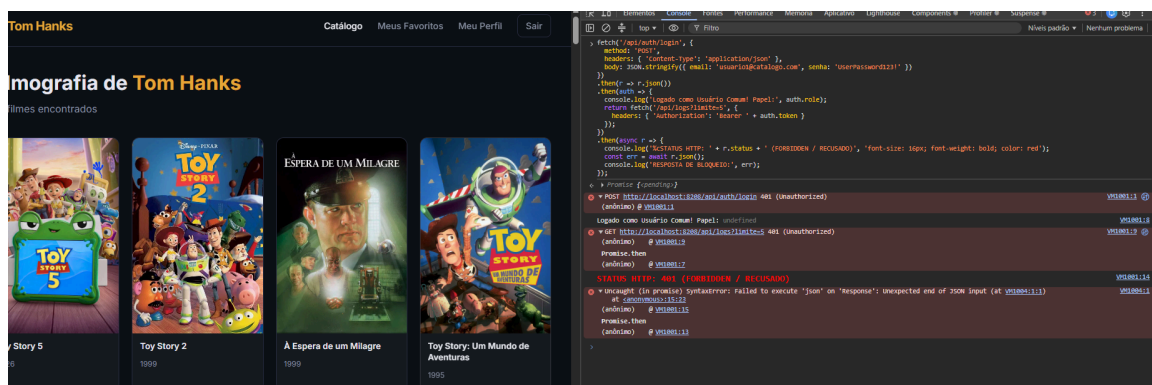


Figura 4: Teste de controle de acesso comprovando recusa e bloqueio frente a usuário sem privilégios administrativos

- ## Hanks

```

    > fetch(`${api}/auth/login`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email: 'admin@catalogo.local', senha: 'AdminPassoword123!!' })
    })
    .then(r => r.json())
    .then(auth => {
      return fetch(`${api}/logs/limited`, {
        headers: { 'Authorization': `Bearer ${auth.token}` }
      });
    })
    .then(async r => {
      console.log('STATUS HTTP: ', +r.status, 'font-size: 16px; font-weight: bold; color: green');
      const logs = await r.json();
      console.table(logs);
    });
  }

  <- Promise.finally(() => {
    statusHttp: 200
  })
  
```

(Índice)	Id	userId	acao	detalhes	ipOrigin	timestamp
0	'1791489814546-e'	7	'login'	'Login realizado po...'	'172.21.0.1'	'2026-10-07T21:15:01.'
1	'1791489793958-f'	7	'login'	'Login realizado po...'	'172.21.0.1'	'2026-10-07T21:14:55.'
2	'1791489863186-g'	7	'login'	'Login realizado po...'	'172.21.0.1'	'2026-10-07T21:14:54.'
3	'1791489576337-h'	7	'tentativa_negada_a...'	'tentativa não auto...'	'172.21.0.1'	'2026-10-07T21:14:51.'
4	'1791489576337-i'	7	'login'	'Login realizado po...'	'172.21.0.1'	'2026-10-07T21:14:51.'

Página 9

```
10) "2026-10-06T23:27:57.4208652Z"
5) 1) "1791329277429-0"
   2) 1) "usuario_id"
      2) "7"
      3) "acao"
      4) "tentativa_idor_bloqueada"
      5) "detalhes"
      6) "Tentativa negada de alterar perfil do usu\xca3\xa1rio 99999"
      7) "ip_origem"
      8) "172.21.0.1"
      9) "timestamp"
      10) "2026-10-06T23:27:57.4208652Z"
6) 1) "1791329172738-0"
   2) 1) "usuario_id"
      2) "7"
      3) "acao"
      4) "login"
      5) "detalhes"
      6) "Login realizado por admin@catalogo.local"
      7) "ip_origem"
      8) "172.21.0.1"
      9) "timestamp"
      10) "2026-10-06T23:26:12.3174354Z"
```

Figura 5.2: Registros persistidos no Redis Streams inspecionados via comando XRANGE no terminal

Ficha 6 — Upload e perfil de usuário

- **Descrição:** Implementação de tela de perfil de usuário com upload de imagem para Object Storage S3 (Garage S3), validação de tipos de arquivo/tamanho máximo e entrega segura via proxy reverso.
- **Data planejada:** 02/10/2026
- **Data realizada:** 02/10/2026 20:20
- **Situação:** Entregue (No prazo)
- **Evidência:** GitHub (Commit, configuração do storage e tela de perfil funcional)
- **Link direto:** Commit 81294e9 — fix(storage): integrar upload e exibição de fotos com Garage S3
- **O que foi feito:** Configuração do cluster leve Garage S3 v1.0 com layout persistente. Implementação de cliente .NET com assinatura AWS Signature Version 4 (SigV4) via HttpClient nativo para contornar incompatibilidades de streaming por chunks em conexões HTTP internas. Criação de endpoint GET /api/profile/photo/{key} integrado ao proxy Nginx /storage/ para entrega das imagens com cache HTTP (max-age=86400).
- **Dificuldades e solução:** O SDK oficial da AWS (AWSSDK.S3) forçava a assinatura proprietária STREAMING-AWS4-HMAC-SHA256-PAYLOAD, rejeitada pelo Garage. A solução definitiva consistiu em implementar o assinador SigV4 diretamente sobre HttpClient, calculando o hash SHA-256 real do arquivo e garantindo compatibilidade total com o storage local.

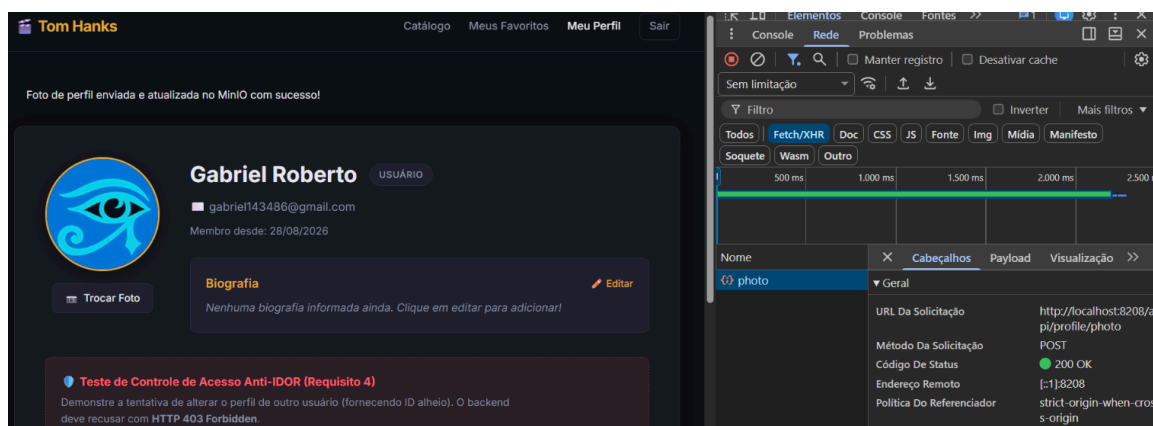


Figura 6: Upload e atualização de foto de perfil no storage de objetos S3 com confirmação visual e HTTP 200 OK

5 Considerações finais

A trajetória percorrida ao longo deste primeiro bimestre proporcionou uma compreensão profunda e prática sobre os desafios reais da arquitetura de sistemas corporativos modernos. A evolução progressiva — partindo de um script monolítico em Flask e culminando em um ecossistema com múltiplos microsserviços containerizados (.NET, Angular, Redis, MariaDB e Garage S3) — demonstrou que a separação de responsabilidades e o isolamento de ambientes trazem enorme robustez operacional, ainda que imponham desafios extras de comunicação e segurança.

O maior obstáculo técnico enfrentado e superado foi a interoperabilidade entre protocolos: desde a sincronização idempotente de migrações em microsserviços compartilhando um banco de dados legado, até a compatibilidade criptográfica de assinaturas AWS SigV4 em armazenamento de objetos auto-hospedado.

Como evolução futura e melhoria de engenharia, seria pertinente a inclusão de um **API Gateway** centralizado com Traefik ou Envoy e a implementação de filas assíncronas com RabbitMQ para publicação desacoplada de eventos de auditoria, garantindo que falhas momentâneas de rede jamais interfiram na latência percebida pelo usuário final.

DECLARAÇÃO DE AUTORIA E INTEGRIDADE

Declaro que este relatório de atividades é de minha exclusiva autoria individual, tendo sido elaborado com base estrita nas implementações desenvolvidas por mim ao longo da disciplina ISW055. Todas as datas, horários de commits e evidências técnicas apontam para o meu repositório público no GitHub e são integralmente verificáveis e auditáveis por qualquer avaliador.

Gabriel Roberto de Assis

São Paulo, 07 de Outubro de 2026